# How browser works behind the scene?

→ DNS lookup, it will locate which server is your domain pointing. This is called DNS lookup and after this step a connection is established the html file is served to browser by server which is used to render page.

→ After a connection is established the html file is served to browser by server which is used to render page.

→ Parsing → In this step, the browser starts parsing the data received from server into DOM (Document object model) and CSSOM (Css object model)

→ Javascript compilation happens here where all js is compiled and interpreted Also in this step accessibility tree is generated.

→ Rendering include style, layout paint and In some cases compositing.

The CSSOM and DOM trees created in parsing step and combined into a render tree which is then used to compute the layout of every visible element, which is painted to screen.

Style → This step combines DOM and CSSOM into a render tree. The Computed style tree, or render tree, construction starts with root of DOM tree, traversing each visible node.

Layout → This step runs layout on the render tree to compute geometry of each node

Paint → This is painting Individual part of screen. It's first time something is visible on screen.

Interactivity → Once the main thread is done painting the page, You would think we would be "all set".

This isn't necessarily the case If load includes javascript that was correctly deferred, and only executed after onload event fires, the main thread might be busy and not available for scrolling, touch and other Interactions.

Time to Interactive (TTI) is the measurement of how long it took first request which led to DNS lookup and TCP connection to when the page is interactive being the point in time after first Contentful paint. when page responds to user interaction within 50 ms.

If main thread is occupied parsing, compiling, executing JS, It is not available therefore not able to respond to user Interactions in a timely (less than 50ms)

# Critical Rendering Path

The critical Rendering path is the sequence of steps the browser goes through to convert html, css and Js to pixels on the screen.

Optimizing critical render path improves render performance

The critical rendering path includes Document Object Model (DOM), CSS Object Model (CSSOM), render tree and layout.

A request for a web page or app starts with an HTML request. The server returns the HTML — response headers and data.

The browser then begins parsing the HTML, converting the receiving bytes to DOM Tree.

The browser initiates request every time to find links to external resources, be it stylesheets, scripts or embedded Image references.

Some requests are blocking, which means the parsing of rest of HTML is halted until the Import asset is handled.

The browser continue to parse HTML making requests and building DOM until it gets to end, at which point it constructs CSS object Model.

With the DOM and CSSOM complete, the browser builds the render tree, computing the styles for all visible content.

After render tree is complete, layout occures, defining location and size of all render tree elements. Once complete, the page is rendered or painted on screen.

Render Tree – The render tree captures both the content and styles: the DOM and CSSOM trees are combined into render tree. To construct the render tree, the broowser check every node, starting from root of DOM Tree, and determines which CSS rules are attached.

The render tree only captures visible content.

Layout - The Layout step determines where and how elements are positioned on the page, determining width and height of each element, and where they are in relation to each other.

Layout performance is impacted by DOM. - the greater the no of nodes, longer layout takes.

Layout can become bottleneck, leading to jank if required during scrolling or other animations

while a 20ms delay on load or orientation change may be fine, it will

lead to jank on animation of scroll.

Any time the render tree is modified, such as by added notes, altered content or updated box model styles on a node. Layout occurs.

To reduce frequency and duration of layout events, batch updates and avoid animating box model properties.

Paint → The last step is painting the pixels to screen.

Once the rendes tree is created and layout occurs, the pixel can be painted to screen.

On load, the entire screen is painted.